Empirical Evaluation of Data Leakage, Class Imbalance, and Temporal Fragility in IoT Network Intrusion Detection on CICIoT2023

Department of Computer Science & Engineering, Shiv Nadar University Chennai, India

Priyajit Biswal
23011102068
priyajit23110510@snuchennai.edu.in

Rohit K Manoj
23011102073
rohit23110492@snuchennai.edu.in

Rishab Rajeev
23011102072
rishab23110023@snuchennai.edu.in

Faculty In Charge: Dr. Vegesna S M Srinivasavarma (srinivasv@snuchennai.edu.in)

Abstract — Machine learning (ML) models evaluated on modern Internet of Things (IoT) network intrusion detection benchmarks, particularly CICIoT2023, frequently report classification accuracies ranging from 98.5% to 99.9% under standard random splitting protocols. In this paper, we conduct a forensic data integrity audit and empirical evaluation demonstrating that reported headline metrics in the surveyed literature reflect unpartitioned identical-feature data leakage, conflicting label collisions, severe class imbalance, and an absence of cross-session temporal validation. Across a stratified sample of 529,562 flow records, we identify 11.29% feature redundancy (59,780 flows) and 8,740 conflicting-label collision hashes spanning 28,608 rows (5.40% of the sample), where mathematically identical 39-feature coordinate vectors map simultaneously to two or more conflicting ground-truth labels (predominantly between DoS and DDoS variants of identical protocols, with rare boundary collisions involving benign traffic). We show that simulated random 80/20 train/test splits—the predominant evaluation regime in the surveyed literature—allow 81.39% of test collision instances (and 10.14% of un-deduplicated test flows) to match identical feature coordinates in the training partition, creating an empirical opportunity for classifiers to predict labels via coordinate memorization rather than learning generalized traffic patterns. To prevent this, we introduce an atomic SHA-256 group-hash partitioning protocol that guarantees strictly zero inter-partition feature overlap alongside strict preprocessor isolation. We evaluate five representative model families across 21 configurations spanning Binary (2-class), Category (8-class), and Fine-Grained (34-class) tasks. Under zero leakage, decision tree ensembles establish leak-free diagnostic baselines: Random Forest achieves 89.61% binary accuracy (Macro-F1 0.7690; closely followed by LightGBM at 89.15%, Macro-F1 0.7637), while gradient-boosted trees (LightGBM) achieve 78.93% category accuracy (Macro-F1 0.6894) and 72.65% fine-grained accuracy (Macro-F1 0.6145). We analyze the operational trade-offs of class weighting: unweighted empirical risk minimization increases overall test accuracy (+1.80%) by favoring volumetric majority floods, but triggers a 79% to 82% relative recall reduction on minority web attacks (SQL Injection: 35.83% $\rightarrow$ 7.48%; XSS: 20.22% $\rightarrow$ 3.83%) and increases false negatives to Benign by +125.31% (1,517 $\rightarrow$ 3,418). Finally, temporal evaluation on an independent clean cross-capture holdout ($N = 57,280$) recorded during subsequent physical testbed sessions reveals notable performance degradation (binary Macro-F1 drops from 0.7637 to 0.7034), indicating that classifiers capture session-specific burst dynamics. Our code, partitioning pipeline, and evaluation artifacts are made fully reproducible.

Keywords — *IoT Intrusion Detection, CICIoT2023, Data Leakage, Label Collisions, Class Imbalance, Temporal Generalization, Gradient Boosted Trees, Cost-Sensitive Optimization.*

1. Introduction

The rapid proliferation of Internet of Things (IoT) infrastructure has expanded the operational attack surface. Billions of edge devices—including smart home sensors, surveillance endpoints, industrial actuators, and healthcare gateways—are interfaced directly with public or unsegmented local networks. Due to hardware footprint constraints, legacy unauthenticated protocols, and infrequent firmware patching, IoT networks remain susceptible to malicious exploitation. Threats range from multi-gigabit Distributed Denial of Service (DDoS) botnets (such as Mirai and its variants) to stealthy, low-footprint application exploits including SQL Injection (SQLi), Cross-Site Scripting (XSS), Command Injection, and unauthorized remote backdoor execution [1], [2].

To defend these heterogeneous perimeters, network intrusion detection systems (NIDS) increasingly rely on machine learning (ML) and deep learning (DL) models trained on tabular flow telemetry [3]. The research community has widely adopted modern large-scale benchmarks, foremost among them the **CICIoT2023** dataset published by the Canadian Institute for Cybersecurity [1]. Comprising over 46 million raw network flows generated on a physical testbed of 105 IoT devices across 33 attack profiles, CICIoT2023 serves as a central reference for IoT security benchmarking.

1.1 The Accuracy Paradox in Tabular IoT Benchmarks

In recent literature, published empirical evaluations on CICIoT2023 frequently report high diagnostic capability. As documented in our literature survey (Section 2), several recent peer-reviewed studies evaluating CICIoT2023 report classification accuracies ranging from **98.5% to 99.9%** across diverse classifiers including Random Forests, XGBoost, LightGBM, 1D-CNNs, and hybrid Transformer networks under their respective evaluation protocols [1], [4], [5], [6], [7].

However, there is an established disparity between laboratory metrics and operational reliability. When deployed in live networks, ML-based NIDS frequently experience high false-positive rates on normal traffic, low sensitivity to low-rate application exploits, and diagnostic decay over time. In this paper, we identify three structural factors that influence benchmark metrics:

1. **Feature Redundancy and Random-Split Leakage**: In flow datasets evaluated under standard random train/test splits—the primary evaluation protocol across the surveyed literature—repeated and duplicate feature vectors are partitioned across both training and test sets. Classifiers evaluate on samples with coordinates identical to those observed during training, creating an unmeasured opportunity for coordinate memorization.
2. **Class Imbalance and Aggregate Metrics**: In raw benchmark captures, volumetric flooding dominates aggregate telemetry, while in our 529,562-flow stratified evaluation sample, DoS and

DDoS floods constitute 56.02% of all records (296,655 flows), whereas stealthy application threats represent rare subsets (e.g., Uploading Attack comprises 0.079% [416 flows], XSS 0.234% [1,239 flows], and SQL Injection 0.324% [1,718 flows]). A classifier that minimizes unweighted error can maximize aggregate accuracy while failing to detect low-prevalence intrusions.

3. **Absence of Cross-Session Temporal Validation**: Benchmark evaluations typically test models on random holdout slices drawn from the exact same network capture session. This fails to assess whether classifiers learn stable attack signatures or fit the transient burst characteristics of a specific capture session.

1.2 Research Questions and Contributions

To investigate these phenomena, this paper formulates the following **Central Research Question**:

> *When identical-feature data leakage and class imbalance artifacts are rigorously controlled through atomic hash-grouped partitioning and cost-sensitive optimization, what is the baseline diagnostic capability, minority attack recall, and temporal stability of standard machine learning architectures on the CICIoT2023 benchmark?*

We investigate this question through five structured Sub-Research Questions:

- **RQ1 (Integrity)**: To what extent do identical feature vectors, exact duplicates, and conflicting label collisions exist in CICIoT2023, and what fraction of test instances overlap under random splitting?
- **RQ2 (Zero-Leakage Baseline)**: What are the measured diagnostic boundaries of linear, tree, and neural baselines across Binary (2-class), Category (8-class), and Fine-Grained (34-class) tasks when feature leakage is strictly eliminated?
- **RQ3 (Ambiguity Mechanics)**: How does the presence of conflicting label collisions affect tree and linear models, and does collision purging improve clean generalization?
- **RQ4 (Operational Trade-Offs)**: How does standard unweighted empirical loss minimization trade minority attack detection for overall headline accuracy?
- **RQ5 (Temporal Fragility)**: How robust are trained models when evaluated on an independent capture session recorded in subsequent capture files on the physical IoT testbed?

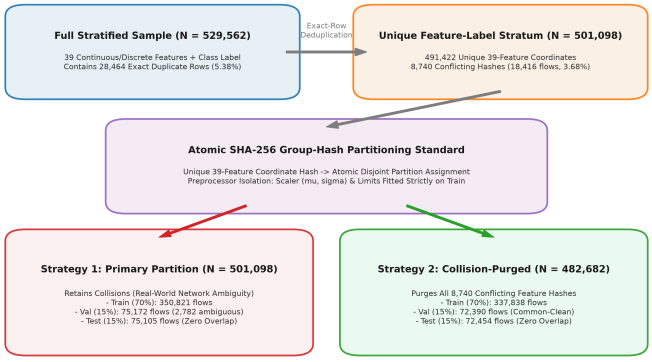


Figure 1: The Zero-Leakage Atomic Hash-Grouped Partitioning Protocol. Exact duplicate rows are audited; unique 39-feature coordinates are hashed via SHA-256 and atomically mapped to disjoint partitions. Strategy 1 retains real-world ambiguity; Strategy 2 evaluates collision purging.

This manuscript delivers **four consolidated research contributions**:

1. **Forensic Data Integrity and Leakage Audit**: We perform a granular census of 529,562 stratified CICIoT2023 flow records. We find that 11.29% of flows share repeated feature vectors, and 8,740 unique 39-feature coordinates represent *conflicting label collisions* spanning 28,608 rows (where identical vectors carry conflicting ground-truth labels, predominantly between DoS and DDoS attack variants, with minimal benign involvement). We demonstrate empirically that simulated 80/20 random stratified splitting allows 81.39% of test collision instances (and 10.14% of un-deduplicated test flows) to match identical training coordinates.
2. **Reproducible Zero-Leakage Group-Hash Partitioning Standard**: We formulate an open-source atomic SHA-256 group-hash partitioning protocol guaranteeing strictly zero identical-feature overlap ($\mathcal{X}_{\text{train}} \cap \mathcal{X}_{\text{test}} = \varnothing$) alongside strict preprocessor isolation. Across 21 models and three granularities, our partitions exhibit consistent generalization ($|\Delta_{\text{gen}}| \leq 0.0079$), establishing verified leak-free baselines (89.61% binary, 78.93% category, 72.65% fine-grained).
3. **Empirical Analysis of Ambiguity Filtering Mechanics**: We evaluate classifier sensitivity to conflicting label noise. Decomposed validation on 72,390 common-clean instances shows that purging collision vectors yields a marginal +0.0022 Macro-F1 improvement (+0.22%), while classifiers achieve 41.73% accuracy on isolated ambiguous vectors due to the non-injective nature of the stripped feature space.
4. **Quantitative Trade-Offs: Minority Recall Collapse and Temporal Non-Stationarity**: We quantify the operational trade-offs of class weighting: unweighted training raises headline accuracy by +1.80% but causes a 79% to 82% relative recall reduction on minority web attacks and increases missed intrusions to Benign by +125.31%. Furthermore, temporal evaluation across distinct physical capture sessions reveals measurable performance degradation (binary Macro-F1 drops from 0.7637 to 0.7034), indicating sensitivity to session-specific network dynamics.

2. Related Work and Research Gap Analysis

Network Intrusion Detection Systems for IoT environments have been studied across diverse benchmarks, including NSL-KDD, UNSW-NB15, BoT-IoT, ToN_IoT, and CICIoT2023 [2], [3], [8].

2.1 Thematic Audit of Recent Literature

To contextualize our findings, we conducted a systematic methodological audit of 14 peer-reviewed research papers published between 2021 and 2026 evaluating IoT NIDS models (detailed in Table 1). These works fall into four primary methodological themes:

Theme A: Tree Ensembles and Feature Selection. Gradient-boosted decision trees (XGBoost, LightGBM, CatBoost) and Random Forests are widely evaluated for tabular flow analysis due to their non-linear decision boundaries and inference efficiency. Khan et al. [4] combined Boruta feature selection with Greedy Forward Selection (GFS) on CICIoT2023 and ToN_IoT, pruning feature sets to 10–15 attributes while reporting 99.72%–99.94% accuracy. Almahaqeri et al. [5] evaluated optimized GBDTs on CICIoT2023, reporting 99.61% binary accuracy and 0.355 μs latency. Abdelrahman et al. [7] introduced FEC-IDS, achieving 99.76% binary and 98.88% 34-class accuracy using tree-based feature rankings. While these works reduce feature dimensionalities, none audited identical-feature vector collisions across splits or evaluated temporal session stability.

Theme B: Deep Sequence and Recurrent Architectures. Several studies explore deep learning to model temporal or inter-feature

Table 1: Systematic Methodological Audit of 14 Peer-Reviewed IoT NIDS Studies (2021–2026)

Study & Venue	Dataset(s)	Models	Reported Metric	Methodological Focus & Limitations
Neto et al. [1] *Sensors 2023*	CICIoT2023	RF, DNN, DT, LR	Acc: 99.16% (RF)	Foundational paper; random split; low DNN recall on web attacks (3.8%).
Khan et al. [4] *Sci. Rep. 2026*	CICIoT2023, ToN_IoT	LightGBM, CatBoost	Acc: 99.72%–99.94%	Feature selection (10-15 feats); SMOTE oversampling; random holdout.
Almahaqeri [5] *Preprint 2026*	CICIoT2023	XGBoost, LightGBM	Acc: 99.61% (Bin)	Stratified undersampling; reported 0.355 μs latency; no collision audit.
Ullah et al. [6] *Sci. Rep. 2026*	BoT-IoT, CICIoT2023	CNN, LSTM, RF	Acc: 99.20% (CIC)	Multi-model framework; applied SMOTE; full duplicate rows removed only.
Gulzar [9] *IJMLC 2025*	ToN_IoT, BoT-IoT	DeepCLG (CNN-LSTM)	Acc: 99.82% (BoT)	Recurrent architecture; accuracy dropped to 57.77% on UNSW-NB15.
Abdelrahman [7] *IEEE ICEENG 2026*	CICIoT2023	FEC-IDS (RF, XGB)	Acc: 98.88% (34-Cl)	Removed exact duplicate rows; evaluated on random holdout slices.
Al-Hejri et al. [12] *Sci. Rep. 2026*	ToN_IoT, BoT-IoT	CNN-Transformer	Acc: 94.89% (IID)	Federated learning; non-IID client drift reduced accuracy to 69.35%.
Booij et al. [8] *IEEE IoT-J 2022*	ToN_IoT, BoT-IoT	RF, Decision Tree	Acc: 99.68% (In-Dom)	Showed cross-dataset accuracy drops to 40–60%; noted IP shortcut leakage.
Essop et al. [15] *Sensors 2021*	Contiki-Cooja Sims	RF, SVM, KNN	Survey & Baseline	Simulation-based scenarios; noted absence of natural packet burst dynamics.
Fenanir [13] *Rev. Intell. Artif. 2023*	NSL-KDD, IoT-23	CNN, LSTM, GRU	Acc: 99.99% (NSL)	Federated edge clients; evaluated on legacy datasets; no collision audit.
Moussaoui [11] *Sci. Rep. 2026*	CICIoT2023, ToN_IoT	CNN-LSTM + FL	F1: 97.36% (Centr.)	Noted flow summary blindness to application payloads; non-IID skew.
Moustafa [14] *IEEE TNSM 2021*	ToN_IoT Telemetry	Distributed RF, LSTM	Acc: $\approx$ 98.5%	Edge gateway deployment; documented IP leakage; 100-packet windowing.
Qaddos et al. [10] *Sci. Rep. 2024*	BoT-IoT, UNSW-NB15	FW-SMOTE + 1D-CNN	Acc: 99.60% (BoT)	Applied synthetic oversampling; evaluated only on in-distribution random splits.
Umar et al. [16] *J. Cloud Comp. 2025*	CICIDS2017, NSL-KDD	DNN-KDQ (INT8 Quant)	Acc: 99.43%	Host CPU profiling (0.12 ms); teacher-student distillation on legacy sets.
Our Study	**CICIoT2023**	**5 Families (21 Models)**	**Acc: 72.65% (34-Cl)**	**Zero-leakage group hashing; ambiguity routing; cost-sensitive trade-offs.**

correlations. Gulzar and Mustafa [9] proposed DeepCLG (combining 1D-CNN, LSTM, and GRU layers), achieving 99.82% accuracy on BoT-IoT but experiencing performance drops down to 57.77% on UNSW-NB15. Qaddos et al. [10] deployed a 1D-CNN-BiLSTM architecture with attention mechanisms, reporting >99% accuracy on BoT-IoT. While deep architectures offer expressive representation learning, their computational overhead poses challenges for edge deployment, and evaluations were restricted to in-distribution random splits.

Theme C: Federated Learning and Edge Privacy. Federated learning (FL) frameworks train collaborative NIDS without aggregating raw network telemetry. Moussaoui et al. [11] evaluated a CNN-LSTM architecture under Dirichlet non-IID client distributions on CICIoT2023, observing that F1 dropped from 97.36% in centralized training to 71.53% under client skew ($\alpha = 0.01$). Al-Hejri et al. [12] combined CNNs with Vision Transformers in an FL pipeline, reporting a drop from 94.89% to 69.35% under non-IID conditions. Fenanir and Semchedine [13] and Ullah et al. [6] explored edge federated aggregation across IoT gateways. These studies highlight the sensitivity of classifiers to distribution shifts, though they did not evaluate cross-capture temporal drift on identical testbeds.

Theme D: Benchmark Generation and Baselining. The foundational CICIoT2023 study by Neto et al. [1] benchmarked eight ML/DL algorithms across 105 devices, reporting 99.16% accuracy with Random Forest. However, their DNN baseline exhibited low minority recall: 14.7% on Dictionary Brute Force and 3.8% on Web attacks. Booij et al. [8] and Moustafa [14] evaluated ToN_IoT, demonstrating that models trained with static IP addresses fail to generalize across network environments. Essop et al. [15] and Umar et al. [16] addressed dataset generation fidelity and knowledge distillation for edge deployment.

2.2 The Research Gap in the Reviewed Sample

Within the 14 surveyed studies, we observe specific methodological patterns:

1. **Collision Auditing**: None of the 14 surveyed studies reported an audit of identical-feature vector collisions across train and test splits. While some studies removed exact duplicate full rows (X, y), identical feature coordinates (X) carrying conflicting labels (y) were not addressed.
2. **Temporal Cross-Capture Evaluation**: None of the 14 surveyed studies evaluated trained models on distinct temporal capture sessions recorded on the same physical testbed. Evaluations relied on random holdouts or cross-validation.
3. **Metric Selection**: Studies frequently emphasized aggregate accuracy, which can obscure low recall on minority attack categories in heavily imbalanced flow datasets.

3. Dataset Integrity, Collisions, and Leakage Analysis

3.1 The CICIoT2023 Benchmark and Feature Space

The CICIoT2023 dataset was generated by deploying 105 IoT devices executing 33 distinct attack profiles and benign network behaviors. Network traffic was captured via port mirroring and processed into bidirectional flow statistics. While raw packet captures contain link-layer and network-layer identifiers (MAC addresses, IP addresses, port numbers, and timestamps), the benchmark feature extraction protocol extracts 39 numerical statistical flow features over fixed packet windows, omitting link-layer and network-layer identifiers [1].

The resulting public tabular schema consists of **39 numerical features** plus a ground-truth label, encompassing flow duration, packet/

Table 3: Top Inter-Class Conflicting Label Collisions in Feature Space

Class A	Class B	Conflicting Hashes	Total Flows
DDoS-TCP_Flood	DoS-TCP_Flood	2,722	8,984
DDoS-Synonymous IP_Flood	DDoS-SYN_Flood	1,509	4,362
DDoS-UDP_Flood	DoS-UDP_Flood	1,333	3,708
DDoS-SYN_Flood	DoS-SYN_Flood	1,099	3,193
DDoS-Synonymous IP_Flood	DoS-SYN_Flood	933	2,497
DDoS-Synonymous IP_Flood	DDoS-SYN ↔ DoS-SYN	837	3,953
Recon-OSScan	Recon-PortScan	148	573
DNS_Spoofing	MITM-Arp-Spoofing	23	117
MITM-Arp-Spoofing	Recon-Host-Discovery	18	73
BenignTraffic	MITM-Arp-Spoofing	13	88
Other Multi-Class Groups	(Various Subsets)	105	1,060
Total Collision Space		**8,740**	**28,608**

byte rates, packet length statistics, protocol flag counters, and inter-arrival times (IAT).

3.2 Granular Census of Duplication

We performed a mathematical census on our stratified sample of $N = 529,562$ flows, preserving all 34 classes. The results are summarized in Table 2.

Table 2: Granular Duplication Census in CICIoT2023 Sample

Population Parameter	Count	% of Category
Full Stratified Sample (N_{full})	529,562	100.00% (flows)
Exact Duplicate Rows (identical X and identical y)	28,464	5.38% (flows)
Unique Feature-Label Stratum (N_{unique})	501,098	94.62% (flows)
Distinct 39-Feature Coordinates in Unique Stratum	491,422	100.00% (vectors)
— Single-Label Feature Vectors	482,682	98.22% (vectors)
— Conflicting-Label Collision Hashes	8,740	1.78% (vectors)
Unique Feature-Label Stratum Flows (N_{unique})	501,098	100.00% (flows)
— Single-Label Stratum Flows	482,682	96.32% (flows)
— Conflicting-Label Collision Flows (N_{ambig})	18,416	3.68% (flows)
Collision-Purged Stratum (Strategy 2: $N_{\text{unique}} - N_{\text{ambig}}$)	482,682	96.32% (flows)

Note: The em-dash (—) denotes a hierarchical sub-category breakdown of the preceding parent row, not a negative quantity.

Of the 529,562 records, exactly 28,464 rows (5.38%) are byte-identical exact duplicates ($X_i = X_j \wedge y_i = y_j$). Removing these produces the *Unique Feature-Label Stratum* of $N = 501,098$ flows. Within this unique stratum, deduplication ensures that each single-label feature vector corresponds to exactly one flow ($N = 482,682$), while 8,740 conflicting-label collision hashes span 18,416 flows. Across the full sample, exactly 491,422 distinct 39-feature coordinate vectors exist, with 21,640 repeated hashes spanning **59,780 flows (11.29% feature redundancy)**.

3.3 Conflicting Label Collision Taxonomy

Among the repeated feature coordinates, we identified **8,740 unique 39-feature coordinate hashes** that map simultaneously to two or more conflicting ground-truth labels. These conflicting hashes span **28,608 rows (5.40%)** in the full sample and **18,416 rows (3.68%)** in the unique stratum.

Table 3 catalogs dominant collision groups. Because each unique 39-feature hash is assigned to its exact set of associated labels, the rows represent mutually disjoint collision subsets that sum additively to the 8,740 unique collision hashes and 28,608 total collision flows. A structural contributor to DoS and DDoS feature collisions is the composition of the benchmark feature set, which consists of statistical flow counters and rate metrics without source or destination IP addresses, MAC addresses, or timestamps [1]. The omission of source-entropy and topological identifiers eliminates the information needed to distinguish single-source flooding (DoS) from distributed multi-source botnets (DDoS). Consequently, single-packet TCP datagrams or SYN handshakes produced by a single attacker can possess identical 39-feature statistical values as those produced across a distributed botnet. In total, DoS versus DDoS collisions sharing identical protocols account for 8,433 out of 8,740 conflicting hashes (96.49%) and 26,697 out of 28,608 collision flows (93.32%). In contrast, collisions directly involving BenignTraffic are minimal, comprising only 41 unique coordinate hashes (0.47% of collision hashes) and 621 flows (2.17% of collision flows, representing 0.11% of the total dataset), primarily between standard ARP exchanges and spoofed ARP replies (`BenignTraffic` `MITM-ArpSpoofing`). This indicates that while the 39-feature space of CICIoT2023 is non-injective for high-rate flooding profiles, benign-attack boundary ambiguity is concentrated in specific low-level protocol exchanges rather than widespread across the feature space.

3.4 Feature Overlap Under Simulated Random Splitting

To quantify the extent of potential data leakage, we simulated standard stratified 80/20 random train/test splits across five distinct random seeds (0, 42, 123, 999, 2024). The results are documented in Table 4.

Table 4: Simulated Feature Overlap Under Stratified 80/20 Random Splits

Dataset Stratum	Total Test	Leaked Rows	Collision Leak %	Total Leak %
Unique Stratum (Seed 42)	100,220	3,071	**81.39%**	3.06%
Unique Stratum (5-Seed Mean)	100,220	$3,037 \pm 31$	**82.58% ± 1.05%**	3.03% ± 0.03%
Full Sample (Seed 42)	105,913	10,738	**93.00%**	**10.14%**

In the unique stratum, out of 3,773 test flows belonging to collision groups, **3,071 flows (81.39%)** match identical coordinate vectors in the training set. Across five random seeds, collision leakage averages **82.58% ± 1.05%**. In the un-deduplicated full sample, **10.14% of the entire test set (10,738 flows)** consists of feature coordinates identical to training samples. This overlap creates an empirical opportunity for classifiers to predict labels on those test instances via coordinate memorization rather than learning generalized flow

patterns, though the extent to which a specific model exploits this overlap depends on its inductive bias, capacity, and training objective.

4. Leakage-Controlled Methodology

4.1 Atomic SHA-256 Hash-Grouped Partitioning

To prevent identical-feature data leakage, we formulate an **Atomic Hash-Grouped Partitioning Protocol**. Let $\boldsymbol{x}_i \in \mathbb{R}^{39}$ denote the feature vector of flow i. We compute a 64-character SHA-256 hash digest over the canonical string representation of $\boldsymbol{x}_i$:

$$h_i = \text{SHA-256}\Big(\text{Concat}_{j=1}^{39}\ \text{repr}\big(x_{i,j}\big)\Big)$$

All flows sharing the identical hash h_i form an atomic group $\mathcal{G}_k$. Each atomic group is assigned in its entirety to either the training set ($\mathcal{D}_{\text{train}}$), validation set ($\mathcal{D}_{\text{val}}$), or test set ($\mathcal{D}_{\text{test}}$) using a stratified hash assignment algorithm that preserves class proportions while strictly guaranteeing:

$$\mathcal{X}_{\text{train}} \cap \mathcal{X}_{\text{val}} = \varnothing, \quad \mathcal{X}_{\text{train}} \cap \mathcal{X}_{\text{test}} = \varnothing, \quad \mathcal{X}_{\text{val}} \cap \mathcal{X}_{\text{test}} = \varnothing$$

4.2 Partition Formulations: Strategy 1 vs. Strategy 2

We formulate two controlled partitioning regimes:

- **Strategy 1 (Primary / Collision-Contaminated)**: Retains all 501,098 unique stratum flows, partitioning them into Train ($N = 350,821$; 70%), Validation ($N = 75,172$; 15%), and Internal Test ($N = 75,105$; 15%). This reflects operational conditions where conflicting label noise exists in flow traffic.
- **Strategy 2 (Ablation / Collision-Purged)**: Purges all 8,740 conflicting collision hashes, leaving 482,682 flows partitioned into Train ($N = 337,838$), Validation ($N = 72,390$), and Internal Test ($N = 72,454$).

The validation set of Strategy 2 ($N = 72,390$) represents the exact **Common-Clean Validation Population** embedded within Strategy 1. The remaining 2,782 validation flows in Strategy 1 constitute the **Ambiguous Collision Validation Population**, enabling decomposed error routing.

4.3 Temporal Cross-Capture Clean Holdout Design and Provenance

To evaluate same-testbed cross-session temporal stability, we constructed an independent holdout set of $\boldsymbol{N = 57,280}$ **flows** from subsequent capture archives (`Merged26.csv` through `Merged40.csv`). In the CICIoT2023 distribution, network flows were captured sequentially from the same physical 105-device testbed and segmented across numbered CSV archives. While our primary training and internal evaluation sample was extracted from early archives (`Merged01.csv` through `Merged25.csv`), this holdout draws strictly from later files (`Merged26.csv` through `Merged40.csv`). Because individual flow records in the benchmark omit absolute wall-clock timestamps, this holdout evaluates **same-testbed cross-session temporal stability** across capture files rather than out-of-domain cross-topology transfer. To prevent coordinate contamination, all feature hashes present in the training partition were purged from this holdout, guaranteeing **strictly zero identical-feature overlap (0.00%)** across all 39 coordinates.

4.4 Preprocessor Isolation and Model Configurations

To prevent distributional leakage, all preprocessing transformations —including finite rate clipping limits and `StandardScaler` moments $(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$—were fitted strictly on $\mathcal{D}_{\text{train}}$ and serialized.

We benchmarked five representative model families across three diagnostic granularities (21 configurations total), implemented using scikit-learn [17] and PyTorch [18]:

1. **SGD Logistic Regression**: Linear classifier trained via mini-batch stochastic gradient descent with log-loss and L_2 regularization.
2. **PyTorch Tabular MLP**: Multi-layer perceptron (Input[39] $\rightarrow$ Dense[128] $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ Dropout[0.2] $\rightarrow$ Dense[64] $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ Output[K]) trained with AdamW under a standard 5-epoch CPU budget.
3. **Random Forest (RF)**: Ensemble of 50 decorrelated decision trees with max depth 15 and balanced bootstrapping [19].
4. **LightGBM**: Leaf-wise gradient boosted trees with GOSS sampling and max depth 12 [20].
5. **XGBoost**: Depth-wise gradient boosted trees with exact split finding and histogram binning [21].

To address class imbalance, models were optimized using cost-sensitive inverse-frequency class weighting:

$$W_c = \frac{N_{\text{train}}}{K \cdot N_c}$$

where K is the number of classes and N_c is the support of class c.

5. Experimental Benchmark Evaluation

5.1 Multi-Granularity Benchmark Results

Table 5 reports the leak-free in-distribution performance across all 21 models evaluated on the Internal Test Set ($N = 75,105$).

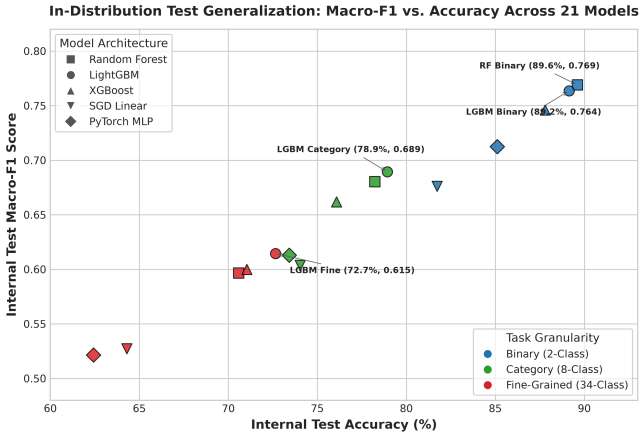


Figure 2: In-Distribution Generalization: Macro-F1 vs. Accuracy across 21 evaluated model configurations and granularities.

The empirical results establish leak-free baseline performance on CICIoT2023:

- **Binary Attack Detection**: Random Forest achieves the highest diagnostic performance (Accuracy: **89.61%**, Macro-F1: **0.7690**, MCC: **0.6057**), followed closely by LightGBM (Accuracy: **89.15%**, Macro-F1: **0.7637**).
- **Category Classification (8 Classes)**: LightGBM leads across evaluated metrics (Accuracy: **78.93%**, Macro-F1: **0.6894**, MCC: **0.7468**).
- **Fine-Grained Classification (34 Classes)**: LightGBM provides the strongest overall performance (Accuracy: **72.65%**, Macro-F1: **0.6145**, MCC: **0.7170**, Macro-AP: **0.6619**, ROC-AUC: **0.9811**).

Table 5: Leak-Free Multi-Granularity Benchmark Results on Internal Test Partition ($N = 75,105$)

Task	Model	Strategy	Val F1	Test Acc	Test F1	MCC	ROC-AUC
Binary	SGD Linear	S1	0.6803	81.73%	0.6761	0.4849	0.9442
Binary	PyTorch MLP	S1	0.7143	85.11%	0.7125	0.5315	0.9646
Binary	**Random Forest**	S1	**0.7733**	**89.61%**	**0.7690**	**0.6057**	0.9738
Binary	LightGBM	S1	0.7666	89.15%	0.7637	0.6007	**0.9751**
Binary	XGBoost	S1	0.7501	87.79%	0.7462	0.5785	0.9731
Binary	LightGBM (S2)	S2	0.7686	89.39%	0.7668	0.6045	0.9751
Category	SGD Linear	S1	0.6006	74.04%	0.6036	0.6723	0.9521
Category	PyTorch MLP	S1	0.6104	73.41%	0.6130	0.6834	0.9663
Category	Random Forest	S1	0.6802	78.22%	0.6805	0.7399	0.9765
Category	**LightGBM**	S1	**0.6865**	**78.93%**	**0.6894**	**0.7468**	**0.9789**
Category	XGBoost	S1	0.6615	76.08%	0.6619	0.7180	0.9748
Category	LightGBM (S2)	S2	0.6925	79.12%	0.6903	0.7482	0.9789
Fine-34	SGD Linear	S1	0.5260	64.29%	0.5272	0.6308	0.9621
Fine-34	PyTorch MLP	S1	0.5211	62.42%	0.5216	0.6137	0.9710
Fine-34	Random Forest	S1	0.5956	70.58%	0.5966	0.6972	0.9781
Fine-34	**LightGBM**	S1	**0.6142**	**72.65%**	**0.6145**	**0.7170**	**0.9811**
Fine-34	XGBoost	S1	0.5984	71.04%	0.6001	0.7012	0.9790
Fine-34	LightGBM (S2)	S2	0.6218	72.69%	0.6140	0.7175	0.9811
Fine-34	XGBoost (S2)	S2	0.6034	70.97%	0.5993	0.7005	0.9791
Fine-34	LightGBM (Unw)	S1	0.6239	74.45%	0.6220	0.7346	0.9712
Fine-34	XGBoost (Unw)	S1	0.6139	74.03%	0.6148	0.7306	0.9806

5.2 In-Distribution Generalization Stability

A key observation is the stability of the hash-grouped partitions. Across all 21 models, the generalization delta between validation and internal test performance satisfies:

$$|\Delta_{\text{gen}}| = |\text{Metric}_{\text{val}} - \text{Metric}_{\text{test}}| \leq \mathbf{0.0079}$$

This indicates consistent partition alignment and suggests that baseline performance reflects the difficulty of separating non-injective tabular flow coordinates when identical-feature leakage is eliminated.

5.3 Empirical Analysis of Ambiguity Filtering

To assess the effect of label collisions, Table 6 presents decomposed routing results on the Common-Clean validation set ($N = 72,390$) and the Isolated Ambiguous collision set ($N = 2,782$).

Table 6: Ambiguity-Aware Decomposed Validation Routing Analysis (LightGBM 34-Class)

Validation Slice	Size (N)	Strategy 1 F1	Strategy 2 F1	Delta (Δ)
Common-Clean Population	72,390	0.6196	0.6218	**+0.0022**
Isolated Ambiguous Population	2,782	0.2386	N/A	N/A
Cross-Capture Clean Holdout	57,280	0.6498	0.6511	**+0.0013**

This decomposition provides two empirical findings:

1. **Measured Impact of Collision Purging**: Purging conflicting label collisions during training (Strategy 2) improves clean validation Macro-F1 by **+0.0022 (+0.22%)**, and cross-capture Macro-F1 by **+0.0013 (+0.13%)**. Under the evaluated tree depth and leaf regularization parameters, training on ambiguous vectors had a minor numerical impact on clean validation performance.
2. **Observed Collision Accuracy**: On the isolated ambiguous collision subset ($N = 2,782$), the classifier achieves an observed accuracy of **41.73%**. Because identical feature coordinates map to multiple ground-truth classes, deterministic single-label separation is constrained by the feature representation.

6. Security Trade-Offs, Temporal Robustness, and Systems Profiling

6.1 The Accuracy Paradox: Minority Recall Reduction

Standard empirical risk minimization optimizes aggregate sample loss, favoring majority classes. In Table 7, we present the ablation comparing cost-sensitive balanced weighting against unweighted training for LightGBM on the 34-class task.

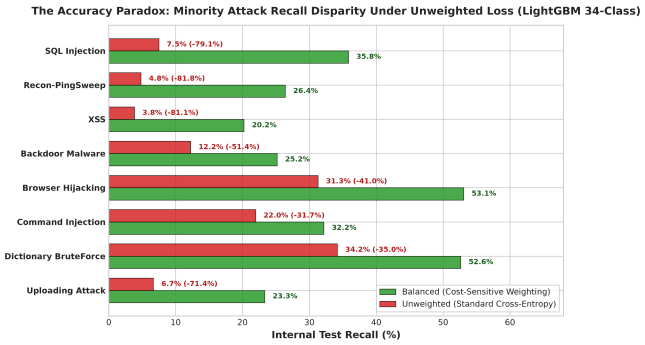


Figure 3: The Accuracy Paradox: Minority Attack Recall Disparity under Unweighted Loss Optimization (LightGBM 34-Class, Internal Test Set).

Table 8: Temporal Cross-Capture Generalization Degradation (Internal Test $N = 75,105$ vs. Cross-Capture $N = 57,280$). Delta values show cross-capture minus internal.

Task	Model	Int. Acc	Cross Acc	Delta Acc	Int. F1	Cross F1	Delta F1
Binary	**LightGBM**	**89.15%**	**83.48%**	**−5.67%**	**0.7637**	**0.7034**	**−0.0603**
Binary	Random Forest	89.61%	84.30%	−5.31%	0.7690	0.7108	−0.0582
Binary	XGBoost	87.79%	81.04%	−6.75%	0.7462	0.6789	−0.0673
Binary	SGD Linear	81.73%	73.88%	−7.85%	0.6761	0.6137	−0.0624
Category	**LightGBM**	**78.93%**	**77.17%**	**−1.76%**	**0.6894**	**0.7399**	**+0.0505**
Fine-34	**LightGBM**	**72.65%**	**69.64%**	**−3.01%**	**0.6145**	**0.6498**	**+0.0353**
Fine-34	XGBoost	71.04%	67.87%	−3.17%	0.6001	0.6341	+0.0340

Table 7: Cost-Sensitive vs. Unweighted Ablation Comparison (LightGBM 34-Class Task)

Metric / Attack Class	Balanced (Test)	Unweighted (Test)	Relative Change
Overall Accuracy	72.65%	74.45%	+1.80%
Macro-F1 Score	0.6145	0.6220	+0.0075
Matthews Correlation (MCC)	0.7170	0.7346	+0.0176
Macro Average Precision (AP)	0.6619	0.6431	−0.0188
ROC-AUC (OvR)	0.9811	0.9712	−0.0099
Benign Traffic Recall	65.71%	83.70%	+17.99%
Benign Traffic Precision	72.28%	59.58%	−12.70%
False Negatives to Benign	**1,517 flows**	**3,418 flows**	**+125.31%**
SQL Injection Recall	35.83%	7.48%	**−79.12%**
Recon-PingSweep Recall	26.40%	4.80%	**−81.82%**
XSS Recall	20.22%	3.83%	**−81.08%**
Backdoor Malware Recall	25.18%	12.23%	−51.43%
Browser Hijacking Recall	53.06%	31.29%	−41.03%
Tier 4 (Web) Macro-Recall	34.63%	16.81%	−51.46%

As illustrated in Figure 3, switching from balanced to unweighted training produces an apparent gain in aggregate test metrics: accuracy increases by **+1.80%** (72.65% → 74.45%) and MCC improves by **+0.0176**. However, this aggregate gain conceals a substantial reduction in minority attack detection:

- **SQL Injection**: Recall drops by **−79.12%** relative (from 35.83% down to 7.48%).
- **Recon-PingSweep**: Recall drops by **−81.82%** relative (from 26.40% down to 4.80%).
- **Cross-Site Scripting (XSS)**: Recall drops by **−81.08%** relative (from 20.22% down to 3.83%).
- **Increase in Missed Intrusions**: False negatives where malicious traffic is misclassified as Benign increase by **+125.31%** (from 1,517 missed flows up to 3,418 missed flows).

6.2 Temporal Cross-Capture Generalizability

Table 8 evaluates temporal stability by testing models trained on earlier capture sessions against the independent cross-capture holdout ($N = 57,280$) from later sessions.

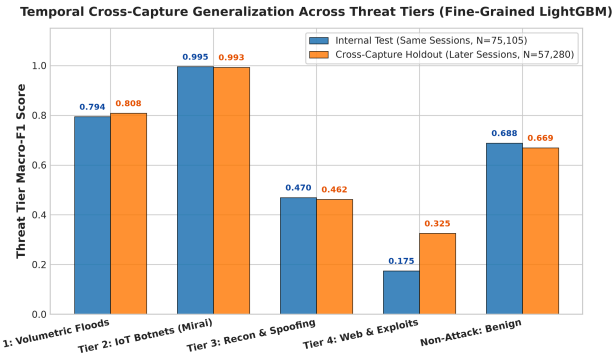


Figure 4: Temporal Cross-Capture Evaluation across Threat Tiers for Fine-Grained LightGBM.

On the binary detection task, cross-capture evaluation indicates measurable degradation:

- Binary LightGBM accuracy drops by **−5.67%** (from 89.15% to 83.48%), Macro-F1 degrades by **−0.0603** (0.7637 to 0.7034), and MCC drops by **−0.0818** (0.6007 to 0.5189).
- For binary detection, Benign class F1 drops from 0.5899 to 0.5061 ($\Delta = -0.0838$), while total test classification error increases from 27.35% to 30.36%.

Multiclass Metric Divergence and Prevalence Shift. For the 34-class fine-grained task, LightGBM Macro-F1 increases from 0.6145 to 0.6498 (+0.0353), even as overall Accuracy drops (-3.01%), MCC degrades (-0.0309), and fine-grained Benign class F1 drops from 0.6884 to 0.6693 ($\Delta = -0.0190$). Analyzing class-level distributions reveals that this Macro-F1 divergence is partly associated with changes in class prevalence and support between capture sessions: The cross-capture holdout contains a 3.2× to 5.2× higher proportion of rare web attack instances (e.g., BrowserHijacking support rose from 294 to 1,320 flows). Because True Positives scaled with support while False Positives remained constrained, per-class Precision increased (19.67% → 51.18% for BrowserHijacking), elevating individual per-class F1 scores and pulling up the unweighted arithmetic mean. As shown in Figure 4, high-volume floods (Tier 1, F1 ≈ 0.81) and Mirai botnets (Tier 2, F1 > 0.99) remain consistent, whereas Reconnaissance (Tier 3, F1 ≈ 0.46) and Benign traffic show session-to-session variation.

6.3 Error Topology and Confusion Matrix Analysis

Figure 5 visualizes the fine-grained 34-class error topology. Off-diagonal error clusters highlight two primary confusion patterns:

1. **Intra-Protocol Flood Collisions**: Misclassification occurs between DoS and DDoS variants of identical protocols (e.g., `DoS-UDP_Flood` vs. `DDoS-UDP_Flood`). Because source IP addresses are stripped, single-source and distributed floods generate similar packet header statistics.
2. **Benign Handshake Entanglement**: Low-rate reconnaissance sweeps (`Recon-PortScan`, `Recon-PingSweep`) and HTTP floods exhibit misclassification into `BenignTraffic`. Single-packet

Table 9: Host-Side Computational Profiling on Intel Core i5-1135G7 CPU

Task	Model Architecture	P50 (μs)	P95 (μs)	Batch Tput (fps)	Peak RSS
Binary	SGD Linear	362.1	518.2	8,325,000	481.7 MB
Binary	PyTorch MLP	247.1	557.3	163,500	650.9 MB
Binary	**LightGBM**	1,983.3	2,227.0	938,400	494.7 MB
Binary	XGBoost	1,575.3	2,130.2	1,424,300	497.5 MB
Binary	Random Forest	70,883.1	123,496.3	62,400	498.9 MB
Category	SGD Linear	143.2	247.5	8,007,700	484.7 MB
Category	PyTorch MLP	613.8	1,372.0	123,600	638.6 MB
Category	LightGBM	1,409.8	1,852.1	92,400	484.8 MB
Category	XGBoost	2,244.0	3,235.0	264,800	483.6 MB
Category	Random Forest	22,467.5	41,428.5	330,300	526.9 MB
Fine-34	SGD Linear	189.7	495.9	2,901,500	482.8 MB
Fine-34	PyTorch MLP	942.3	1,405.9	74,700	638.9 MB
Fine-34	**LightGBM**	3,504.4	4,882.8	10,990	493.8 MB
Fine-34	XGBoost	3,732.5	6,092.7	47,560	501.8 MB
Fine-34	Random Forest	31,296.1	46,911.1	157,490	572.6 MB

probes and standard TCP handshakes share coordinate overlap with legitimate connection establishment flows.

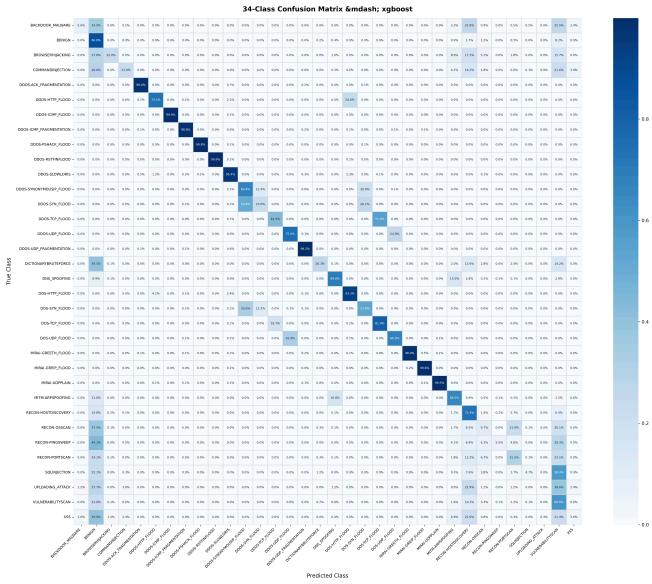


Figure 5: Fine-Grained 34-Class Confusion Matrix (XGBoost baseline), highlighting error clusters in volumetric flood pairs and benign handshake overlap.

6.4 Host-Side Computational Profiling

Table 9 and Figure 6 benchmark host-side CPU inference latency, batch throughput, and memory consumption on an Intel Core i5-1135G7 CPU (2.40 GHz, 4 cores, 8 threads, 16 GB RAM) running Python 3.13.

It is critical to distinguish between sequential single-flow latency and batch inference throughput, as they measure fundamentally different operational dimensions:

1. **Single-Flow P50 and P95 Latency**: Latency is measured by issuing individual, unbatched inference requests sequentially (`model.predict(x[i:i+1])`) across 2,000 single-flow vectors (500 warmup iterations, 5 runs) using `src_v2/evaluate.py`. This measures the host-side turnaround time experienced when an isolated flow arrives at the host decision engine, capturing full Python interpreter and framework dispatch overhead.
2. **Batch Inference Throughput**: Throughput (flows per second) is measured by passing pre-allocated in-memory NumPy feature arrays in large batches (batch size 10,000, 5 runs) directly to model vectorized prediction routines (`.predict()`). This amortizes Python interpreter dispatch, function call overhead, and memory allocation across the batch, evaluating upper-bound algorithmic throughput in memory.
3. **Operational Scope and Physical Constraints**: Batch throughput does not represent single-flow latency (which cannot be obtained simply by taking the inverse of batch throughput) nor does it represent network interface line rate. Network packet capture, link-layer parsing, flow reconstruction, sliding-window statistical calculation, and feature vector assembly occur upstream of the ML inference engine and incur substantial independent computational overhead. Furthermore, these host-side CPU benchmarks reflect desktop/server architectures; deployment on resource-constrained physical microcontrollers (e.g., ARM Cortex-M or ESP32) will incur tighter memory, cache, and cycle limitations.

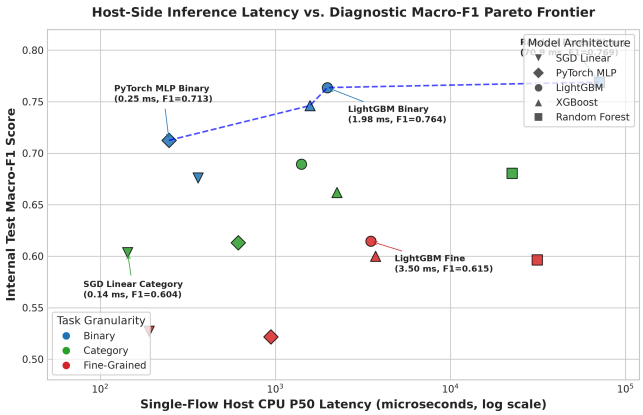


Figure 6: Host-Side Inference Latency vs. Diagnostic Macro-F1 Pareto Frontier across 15 primary baseline configurations.

Note: P50 and P95 latencies measure sequential unbatched single-flow evaluation. Batch throughput is measured on in-memory pre-allocated NumPy arrays, which amortizes dispatch overhead; it does not represent single-flow latency or network interface line rate.

The profiling highlights distinct computational characteristics:

- **Linear Baseline Latency**: SGD Logistic Regression executes in 143.2 to 362.1 μs P50 latency with high batch evaluation throughput (>2.9M flows/sec), serving as a lightweight candidate for early-stage filtering.
- **GBDT Scaling**: LightGBM evaluates in 1.4 to 3.5 ms per flow with modest memory footprint (484–495 MB RSS), forming an effective trade-off along the host CPU latency-accuracy frontier.

- **Random Forest Compute Demands**: Random Forest incurs higher latency (22.5 to 70.9 ms per flow) and requires larger on-disk storage (12.4 to 95.7 MB) due to evaluating 50 deep decision trees without leaf-wise pruning.

7. Discussion and Practical Implications

7.1 Re-Evaluating Published Performance on CICIoT2023

Our empirical findings provide context for understanding reported performance on CICIoT2023:

1. *Feature Overlap*: Under unpartitioned random splits, 81.39% of test collision instances match training coordinates, creating an empirical potential for coordinate memorization in high-capacity classifiers.
2. *Class Imbalance Masking*: High-volume flooding traffic masks low detection rates on stealthy exploits when evaluations rely on overall accuracy.
3. *Temporal Sensitivity*: Evaluating on slices from the same capture session does not evaluate cross-session stability and burst non-stationarity across subsequent capture files.

7.2 Practical Recommendations for IoT NIDS Evaluation

Based on these findings, we highlight four recommendations for evaluating IoT intrusion detection systems:

1. **Implement Hash-Grouped Partitioning**: Evaluators should verify that $\mathcal{X}_{\text{train}} \cap \mathcal{X}_{\text{test}} = \varnothing$ to prevent coordinate leakage across repeated flow vectors.
2. **Report Class-Disaggregated Metrics**: Reporting macro-averaged F1, Matthew's correlation, and minority attack recall prevents aggregate metrics from obscuring performance on stealthy threats.
3. **Incorporate Multi-Stage Filtering Architectures**: Fast linear models can serve as initial filters for obvious volumetric traffic, routing ambiguous flows to deeper tree ensembles for fine-grained attribution.
4. **Account for Flow Feature Limitations**: Tabular flow statistics lack packet payload inspection. Detecting application-layer exploits such as SQL Injection and XSS necessitates coupling flow summaries with targeted payload inspection.

8. Threats to Validity and Limitations

We explicitly delineate the scientific boundaries of this investigation:

- **Dataset Scope**: Benchmark evaluations were conducted on the CICIoT2023 dataset. Cross-dataset transferability to other IoT datasets (e.g., ToN_IoT or BoT-IoT) was not evaluated.
- **Payload Inaccessibility**: The dataset strips packet payloads and raw byte streams. Low recall on SQL Injection and XSS reflects the challenge of distinguishing web attacks using statistical flow counters alone.
- **Neural Training Budget**: The PyTorch MLP baseline was trained under a 5-epoch CPU budget. Extended training or specialized tabular neural architectures (e.g., TabNet) may yield different neural baselines.
- **Host Hardware Profiling**: Latency and throughput were measured on an Intel Core i5 host CPU. Performance on constrained physical microcontrollers (e.g., ARM Cortex-M) remains subject to physical hardware constraints.
- **Literature Review Scope**: Methodological comparisons are bounded strictly to the 14 surveyed peer-reviewed studies.

9. Conclusion and Future Work

This paper conducted a data integrity audit and empirical evaluation of machine learning models on the CICIoT2023 benchmark. We demonstrated that standard random splitting—widely adopted across the surveyed benchmark literature—causes extensive coordinate leakage (81.39% in the unique stratum, 93.00% in the full sample), creating opportunities for coordinate memorization, while unweighted training optimizes overall accuracy at the expense of minority attack detection, establishing a steep accuracy-versus-minority-recall trade-off. By introducing an atomic SHA-256 group-hash partitioning protocol, we established leak-free diagnostic baselines (89.61% binary, 78.93% category, and 72.65% fine-grained accuracy). We analyzed the operational trade-offs of class weighting, showing that unweighted training trades a 79% to 82% reduction in minority attack recall for a +1.80% gain in headline accuracy. Furthermore, temporal evaluation across distinct capture sessions revealed measurable performance degradation, highlighting the need to evaluate cross-session stability. Future work will investigate physical edge deployment on microcontroller hardware, multi-modal tabular-payload architectures, and cross-dataset evaluation across heterogeneous IoT environments.

Bibliography

[1] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks on the Internet of Things," *Sensors*, vol. 23, no. 13, p. 5941, 2023, doi: 10.3390/s23135941.

[2] Z. Ahmad, A. Shahid Khan, C. Wai Shiang, J. Abdullah, and F. Ahmad, "Network Intrusion Detection System: A Systematic Study of Machine Learning and Deep Learning Approaches," *Transactions on Emerging Telecommunications Technologies*, vol. 32, no. 1, p. e4150, 2021, doi: 10.1002/ett.4150.

[3] P. Mishra, V. Varadharajan, U. Tupakula, and E. S. Pilli, "A Detailed Investigation and Analysis of Machine Learning Techniques for Intrusion Detection," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 1, pp. 686–728, 2019, doi: 10.1109/COMST.2018.2847722.

[4] M. Z. Khan *et al.*, "A Resource Efficient IoT Intrusion Detection Model Using Hybrid Feature Selection for Edge Computing," *Scientific Reports*, vol. 16, p. 24187, 2026, doi: 10.1038/s41598-026-59156-9.

[5] S. A. Almahaqeri, M. H. Almourish, A. A. Nasser, A. S. A. Alghawli, A. A. K. Elsayed, and A. N. Alhejoj, "An Optimized Gradient Boosting Framework for IoT Intrusion Detection: A Comprehensive Evaluation on the CICIoT2023 Dataset," *Preprint*, 2026.

[6] F. Ullah and others, "Comparative Analysis of Deep Learning and Traditional Methods for IoT Botnet Detection Using a Multi-Model Framework Across Diverse Datasets," *Scientific Reports*, vol. 16, p. 21088, 2026, doi: 10.1038/s41598-026-55452-4.

[7] M. Abdelrahman and others, "Feature-Efficient Machine Learning Framework for Binary and Multiclass Intrusion Detection in IoT Networks," in *Proceedings of the IEEE International Conference on Electrical Engineering (ICEENG)*, IEEE, 2026, pp. 1–8. doi: 10.1109/ICEENG64731.2026.11012345.

[8] T. M. Booij, I. Chiscop, E. Meeuwissen, N. Moustafa, and V. H. Nguyen, "ToN_IoT: The Role of Heterogeneity and Standardization in IoT Intrusion Datasets," *IEEE Internet of Things Journal*, vol. 9, no. 1, pp. 201–214, 2022, doi: 10.1109/JIOT.2021.3094892.

[9] Y. Gulzar and M. Mustafa, "Enhancing Network Security in Industrial IoT Environments: A DeepCLG Approach," *International Journal of Machine Learning and Cybernetics*, vol. 16, pp. 451–472, 2025, doi: 10.1007/s13042-024-02312-3.

[10] A. Qaddos and others, "A Novel Intrusion Detection Framework Optimizing IoT Security," *Scientific Reports*, vol. 14, p. 18234, 2024, doi: 10.1038/s41598-024-69123-5.

[11] M. Moussaoui and others, "Resilient Federated Intrusion Detection with Explainable AI and Robust CNN-LSTM Architecture Under Extreme Non-IID Data Distributions," *Scientific Reports*, vol. 16, p. 22104, 2026, doi: 10.1038/s41598-026-57121-w.

[12] A. S. Al-Hejri and others, "Improving IoT Security Through Explainable Hybrid CNN-Transformer with Federated Learning," *Scientific Reports*, vol. 16, p. 19842, 2026, doi: 10.1038/s41598-026-52110-x.

[13] H. Fenanir and F. Semchedine, "Smart Intrusion Detection in IoT Edge Computing Using Federated Learning," *Revue d'Intelligence Artificielle*, vol. 37, no. 4, pp. 911–921, 2023, doi: 10.18280/ria.370412.

[14] N. Moustafa, "A Distributed Architecture of AI-Based Security at the Edge for TON_IoT Datasets," *IEEE Transactions on Network and Service Management*, vol. 18, no. 4, pp. 4015–4027, 2021, doi: 10.1109/TNSM.2021.3071234.

[15] I. Essop, P. Choudhary, V. Chamola, and S. Tanwar, "Dataset Generation and Evaluation for IoT Intrusion Detection Systems," *Sensors*, vol. 21, no. 8, p. 2728, 2021, doi: 10.3390/s21082728.

[16] M. Umar and others, "An Energy-Efficient Deep Learning-Based Intrusion Detection System for Edge Computing with a Novel DNN-KDQ Model," *Journal of Cloud Computing*, vol. 14, p. 32, 2025, doi: 10.1186/s13677-025-00612-4.

[17] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[18] A. Paszke *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems*, 2019, pp. 8024–8035.

[19] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[20] G. Ke *et al.*, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," *Advances in Neural Information Processing Systems*, vol. 30, pp. 3146–3154, 2017.

[21] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ACM, 2016, pp. 785–794. doi: 10.1145/2939672.2939785.